beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_weather = pd.read_excel("/content/drive/MyDrive/capstone/datasets/3_weather.xlsx")
```

```
df_weather.head()
```

	age	age_group	gender	marital_status	profession_group	profession_description	workplace	religion	financial_condition	hometown	...	temp_max	air_pressure	air_hum
0	NaN	teen	male	NaN	worker	he was an plumber	rural	muslim	indebted	brahmanbaria	...	299.15	1008.0	
1	20.0	teen	male	rejected	NaN	NaN	NaN	muslim	NaN	sylhet	...	295.57	1012.0	
2	14.0	teen	female	single	student	student of class nine	urban	muslim	NaN	manikganj	...	301.15	1004.0	
3	NaN	NaN	female	rejected	student	student of Bangla Department	urban	muslim	NaN	bogura	...	299.34	1003.0	
4	18.0	teen	female	relationship	student	studied fashion design at a private university	urban	muslim	NaN	dhaka	...	300.15	1004.0	

5 rows × 30 columns

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Get the data types of each column
    data_types = df.dtypes

    # Create a DataFrame for the missing data summary including data types
```

```
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values,
        'Data Type': data_types.values
    })

    # Define the four groups based on missing percentage
    group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage', 'Data Type']]
    group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage'
    group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage'
    group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage', 'Data Type']]

    # Sort each group by missing percentage in descending order (most missing to least missing)
    group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
    group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)


    # Rename columns for clarity
    group_1.columns = ['Column', 'Missing %', 'Data Type']
    group_2.columns = ['Column', 'Missing %', 'Data Type']
    group_3.columns = ['Column', 'Missing %', 'Data Type']
    group_4.columns = ['Column', 'Missing %', 'Data Type']

    # Display each group one by one, from most missing to least missing
    print("Group 1: Columns with >70% missing data")
    print(group_1.to_string(index=False))
    print("\nGroup 2: Columns with >40% to ≤70% missing data")
    print(group_2.to_string(index=False))
    print("\nGroup 3: Columns with >20% to ≤40% missing data")
    print(group_3.to_string(index=False))
    print("\nGroup 4: Columns with ≤20% missing data")
    print(group_4.to_string(index=False))
```

```
missing_data_report(df_weather)
```

```
Group 1: Columns with >70% missing data
Column  Missing % Data Type
  coco      100.0   float64

Group 2: Columns with >40% to ≤70% missing data
                Column  Missing % Data Type
          air_humidity  53.061224   float64
            feels_like  53.061224   float64
            clouds_sky  53.061224   float64
              wind_deg  53.061224   float64
          weather_main  52.999382    object
   weather_description  52.999382    object
profession_description  49.412492    object
   financial_condition  45.516388    object

Group 3: Columns with >20% to ≤40% missing data
        Column  Missing % Data Type
     workplace  37.909709    object
  air_pressure  30.426716   float64
    wind_speed  30.179344   float64
```

```
      temp_min  29.870130   float64
      temp_max  29.560915   float64
marital_status  23.933210    object

Group 4: Columns with ≤20% missing data
            Column  Missing % Data Type
  profession_group  18.923933    object
            reason  16.821274    object
reason_description  16.635745    object
               age  12.739641   float64
              time  12.059369    object
         time_hour  12.059369    object
         age_group   7.421150    object
         longitude   6.493506   float64
          latitude   6.493506   float64
         unix_time   5.009276   float64
            method   2.906617    object
          religion   2.411874    object
      suicide_date   2.040816    object
          hometown   1.669759    object
            gender   0.432900    object
```

```
df_weather.columns
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
       'profession_description', 'workplace', 'religion',
       'financial_condition', 'hometown', 'latitude', 'longitude', 'reason',
       'reason_description', 'time', 'method', 'suicide_date', 'unix_time',
       'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
       'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
       'weather_description', 'time_hour', 'coco'],
      dtype='object')
```

```
def convert_time_hour_to_int(df, column='time_hour'):
    df[column] = df[column].apply(
        lambda x: int(x.split(':')[0]) if isinstance(x, str) and ':' in x else pd.NA
    ).astype('Int64')
    return df
```

```
df_weather = convert_time_hour_to_int(df_weather)
```

```
# Step 1: Convert to datetime
df_weather['suicide_date'] = pd.to_datetime(df_weather['suicide_date'], format='%d/%m/%Y', errors='coerce')

# Step 2: Extract parts as integers, preserving NaN
df_weather['suicide_day'] = df_weather['suicide_date'].dt.day.astype('Int64')
df_weather['suicide_month'] = df_weather['suicide_date'].dt.month.astype('Int64')
df_weather['suicide_year'] = df_weather['suicide_date'].dt.year.astype('Int64')
df_weather['suicide_week'] = df_weather['suicide_date'].dt.isocalendar().week.astype('Int64')
df_weather['suicide_weekday'] = df_weather['suicide_date'].dt.day_name()
```

```
print(df_weather.columns)
print(df_weather.shape)
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
       'profession_description', 'workplace', 'religion',
```

```
       'financial_condition', 'hometown', 'latitude', 'longitude', 'reason',
       'reason_description', 'time', 'method', 'suicide_date', 'unix_time',
       'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
       'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
       'weather_description', 'time_hour', 'coco', 'suicide_day',
       'suicide_month', 'suicide_year', 'suicide_week', 'suicide_weekday'],
      dtype='object')
(1617, 35)
```

```
columns_to_delete = [
    'coco', 'profession_description', 'reason_description', 'suicide_date', 'unix_time'
]

# reasons:-----------------------------------------------------------------------------
# coco: all null values
# profession_description: long text
# reason_description: long text
# suicide_date: used for creating 'suicide_day', 'suicide_month', 'suicide_year', 'suicide_week', 'suicide_weekday' no longer needed
# unix_time: used for getting weather info no longer needed

# Select the columns to delete and display 5 random values
display(df_weather[columns_to_delete].sample(5))
```

	coco	profession_description	reason_description	suicide_date	unix_time
922	NaN	housewife	family feud	2022-05-26	1.653599e+09
1292	NaN	NaN	NaN	NaT	NaN
188	NaN	NaN	He had been mentally disturbed for sometime du...	2020-01-11	1.604189e+09
936	NaN	housewife	family feud	2022-04-06	1.649232e+09
96	NaN	NaN	Family feud	2020-06-11	1.604621e+09

```
# Drop the columns
df_weather = df_weather.drop(columns=columns_to_delete)
```

Exporting non encoded

```
df_non_encoded = df_weather.copy()

# Drop the unnecessary columns from the copied DataFrame
df_non_encoded = df_non_encoded.drop(columns='time_hour')

print("Created a new DataFrame 'df_non_encoded' without columns.")
display(df_non_encoded.head())
```

```
Created a new DataFrame 'df_non_encoded' without columns.
```

	age	age_group	gender	marital_status	profession_group	workplace	religion	financial_condition	hometown	latitude	...	wind_speed	wind_deg	clouds_sky	weather_mai
0	NaN	teen	male	NaN	worker	rural	muslim	indebted	brahmanbaria	23.960600	...	2.10	320.0	40.0	haz
1	20.0	teen	male	rejected	NaN	NaN	muslim	NaN	sylhet	24.899220	...	1.39	116.0	33.0	cloud
2	14.0	teen	female	single	student	urban	muslim	NaN	manikganj	23.832984	...	3.60	190.0	75.0	drizzl
3	NaN	NaN	female	rejected	student	urban	muslim	NaN	bogura	24.850066	...	2.64	186.0	100.0	rai
4	18.0	teen	female	relationship	student	urban	muslim	NaN	dhaka	23.764386	...	1.99	154.0	40.0	haz

5 rows × 29 columns

```
df_non_encoded.columns
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
       'workplace', 'religion', 'financial_condition', 'hometown', 'latitude',
       'longitude', 'reason', 'time', 'method', 'feels_like', 'temp_min',
       'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg',
       'clouds_sky', 'weather_main', 'weather_description', 'suicide_day',
       'suicide_month', 'suicide_year', 'suicide_week', 'suicide_weekday'],
      dtype='object')
```

New Section

```
# from google.colab import files
# filename = '4_non_encoded.xlsx'
# df_non_encoded.to_excel(filename, index=False)
# files.download(filename)
```

```
categorical_columns = [
    'age_group', 'gender', 'marital_status', 'profession_group', 'workplace', 'religion', 'financial_condition',
    'hometown', 'reason', 'method', 'weather_main', 'weather_description', 'suicide_weekday',
]

numerical_columns = [
    'age', 'latitude', 'longitude', 'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
    'wind_speed', 'wind_deg', 'clouds_sky', 'time_hour', 'suicide_day',
    'suicide_month', 'suicide_year', 'suicide_week',
]

not_used_column = ['time']
```

```
# Display data types for categorical columns
print("Data types for categorical columns:")
print(df_weather[categorical_columns].dtypes)
print(len(categorical_columns))

print("\nData types for numerical columns:")
```

```
print(df_weather[numerical_columns].dtypes)
print(len(numerical_columns))
```

```
Data types for categorical columns:
age_group              object
gender                 object
marital_status         object
profession_group       object
workplace              object
religion               object
financial_condition    object
hometown               object
reason                 object
method                 object
weather_main           object
weather_description    object
suicide_weekday        object
dtype: object
13

Data types for numerical columns:
age              float64
latitude         float64
longitude        float64
feels_like       float64
temp_min         float64
temp_max         float64
air_pressure     float64
air_humidity     float64
wind_speed       float64
wind_deg         float64
clouds_sky       float64
time_hour          Int64
suicide_day        Int64
suicide_month      Int64
suicide_year       Int64
suicide_week       Int64
dtype: object
16
```

Encoding

```
# Iterate through the categorical columns and print unique value counts
for col in categorical_columns:
    unique_count = df_weather[col].nunique(dropna=False) # Include NaN in the count
    print(f"{col} ====== unique values (including NaN): {unique_count}")
```

```
age_group ====== unique values (including NaN): 8
gender ====== unique values (including NaN): 4
marital_status ====== unique values (including NaN): 9
profession_group ====== unique values (including NaN): 36
workplace ====== unique values (including NaN): 4
religion ====== unique values (including NaN): 8
financial_condition ====== unique values (including NaN): 6
hometown ====== unique values (including NaN): 151
reason ====== unique values (including NaN): 37
method ====== unique values (including NaN): 13
weather_main ====== unique values (including NaN): 9
```

```
weather_description ====== unique values (including NaN): 16
suicide_weekday ====== unique values (including NaN): 8
```

```
df_weather = df_weather.rename(columns={'time_hour': 'time_encoded'})
print("Column 'time_hour' renamed to 'time_encoded'.")
```

```
Column 'time_hour' renamed to 'time_encoded'.
```

```
# Define ordinal mapping
age_mapping = {'baby': 0, 'child': 1, 'teen': 2, 'young': 3, 'adult': 4, 'middle-aged': 5, 'old': 6}
financial_condition_mapping = {'indebted': 0, 'lower': 1, 'middle': 2, 'upper_middle': 3, 'upper': 4}
weather_main_mapping = {'clear': 0, 'partly cloudy': 1, 'clouds': 2, 'mist': 3, 'haze': 4, 'drizzle': 5, 'rain': 6, 'thunderstorm': 7, }
suicide_weekday_mapping = {'Monday': 0, 'Tuesday': 1, 'Wednesday': 2, 'Thursday': 3, 'Friday': 4, 'Saturday': 5, 'Sunday': 6 }
```

```
# Apply the mapping and convert to nullable integers
df_weather['age_group_encoded'] = df_weather['age_group'].map(age_mapping).astype('Int64')
df_weather['financial_condition_encoded'] = df_weather['financial_condition'].map(financial_condition_mapping).astype('Int64')
df_weather['weather_main_encoded'] = df_weather['weather_main'].map(weather_main_mapping).astype('Int64')
df_weather['suicide_weekday_encoded'] = df_weather['suicide_weekday'].map(suicide_weekday_mapping).astype('Int64')
```

```
display(df_weather['suicide_weekday_encoded'].head(5))
```

	suicide_weekday_encoded
0	4
1	5
2	4
3	3
4	4

dtype: Int64

```
frequency_cols = ['profession_group', 'hometown', 'reason', 'method', 'weather_description', 'weather_main', 'marital_status']

for col in frequency_cols:
    freq_encoding = df_weather[col].value_counts(normalize=True)
    df_weather[f'{col}_encoded'] = df_weather[col].map(freq_encoding)
```

```
display(df_weather[[f'{col}_encoded' for col in frequency_cols]].sample(5))
```

	profession_group_encoded	hometown_encoded	reason_encoded	method_encoded	weather_description_encoded	weather_main_encoded	marital_status_encoded
235	0.225019	0.023899	0.019331	0.749682	0.178947	0.380263	0.491057
1570	0.225019	0.048428	NaN	0.749682	NaN	NaN	0.491057
715	0.440122	0.011950	0.294424	0.749682	0.106579	0.380263	0.444715
1070	0.440122	0.005660	0.008178	0.749682	NaN	NaN	0.444715
1494	0.225019	0.023899	NaN	0.177070	NaN	NaN	0.491057

```
print("Count of each unique category in 'religion':")
display(df_weather['religion'].value_counts(dropna=False))
```

```
Count of each unique category in 'religion':
```

religion	count
muslim	1363
hindu	197
NaN	39
buddhist	7
christian	6
chakma	2
santal	2
marma	1

dtype: int64

```
df_weather['religion'] = df_weather['religion'].replace(['marma', 'santal', 'chakma'], 'indigenous')
print("Categories 'marma', 'santal', and 'chakma' in 'religion' column converted to 'indigenous'.")
display(df_weather['religion'].value_counts(dropna=False))
```

```
Categories 'marma', 'santal', and 'chakma' in 'religion' column converted to 'indigenous'.
```

religion	count
muslim	1363
hindu	197
NaN	39
buddhist	7
christian	6
indigenous	5

dtype: int64

```
one_hot_cols = ['gender', 'workplace', 'religion']

for col in one_hot_cols:
    # Step 1: Create dummies (include dummy_na just to get consistent columns)
    dummies = pd.get_dummies(df_weather[col], prefix=col + '_encoded', dummy_na=False)

    # Step 2: Identify where original values are missing
    nan_mask = df_weather[col].isna()
```

```
    # Step 3: For those rows, set all one-hot encoded columns to NaN
    dummies.loc[nan_mask] = np.nan

    # Step 4: Add back to main dataframe
    df_weather = pd.concat([df_weather, dummies], axis=1)
```

```
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
/tmp/ipython-input-3227143850.py:11: FutureWarning: Setting an item of incompatible dtype is deprecated and will raise an error in a future version of pandas. Value 'nan' |
  dummies.loc[nan_mask] = np.nan
```

```
# Get all columns that have '_encoded_' in their name (assuming this pattern for all one-hot encoded columns)
one_hot_encoded_cols = [col for col in df_weather.columns if '_encoded_' in col]

# Display 5 sample rows from these columns
display(df_weather[one_hot_encoded_cols].head())
```

	gender_encoded_female	gender_encoded_male	gender_encoded_third gender	workplace_encoded_rural	workplace_encoded_semi-urban	workplace_encoded_urban	religion_encoded_buddhist	re
0	False	True	False	True	False	False	False	
1	False	True	False	NaN	NaN	NaN	False	
2	True	False	False	False	False	True	False	
3	True	False	False	False	False	True	False	
4	True	False	False	False	False	True	False	

```
column_to_skip = [
    'age_group', 'gender', 'marital_status', 'profession_group', 'workplace', 'religion', 'financial_condition',
    'hometown', 'reason', 'method', 'weather_main', 'weather_description', 'suicide_weekday', 'time',
]
```

```
df_encoded = df_weather.copy()

# Drop the categorical columns from the copied DataFrame
```

```
df_encoded = df_encoded.drop(columns=column_to_skip)

print("Created a new DataFrame 'df_encoded' excluding categorical columns.")
display(df_encoded.head())
```

```
Created a new DataFrame 'df_encoded' excluding categorical columns.
```

	age	latitude	longitude	feels_like	temp_min	temp_max	air_pressure	air_humidity	wind_speed	wind_deg	...	gender_encoded_male	gender_encoded_third gender	workplace_encode
0	NaN	23.960600	91.119089	302.86	299.15	299.15	1008.0	83.0	2.10	320.0	...	True	False	
1	20.0	24.899220	91.868527	299.17	295.57	295.57	1012.0	96.0	1.39	116.0	...	True	False	
2	14.0	23.832984	89.966688	304.33	301.15	301.15	1004.0	78.0	3.60	190.0	...	False	False	
3	NaN	24.850066	89.372843	303.67	299.34	299.34	1003.0	91.0	2.64	186.0	...	False	False	
4	18.0	23.764386	90.389014	304.49	300.15	300.15	1004.0	83.0	1.99	154.0	...	False	False	

5 rows × 37 columns

```
missing_data_report(df_encoded)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %, Data Type]
Index: []

Group 2: Columns with >40% to ≤70% missing data
                     Column  Missing % Data Type
                 feels_like  53.061224   float64
              air_humidity  53.061224   float64
                  wind_deg  53.061224   float64
                clouds_sky  53.061224   float64
       weather_main_encoded  52.999382   float64
weather_description_encoded  52.999382   float64
financial_condition_encoded  45.516388     Int64

Group 3: Columns with >20% to ≤40% missing data
                      Column  Missing % Data Type
workplace_encoded_semi-urban  37.909709    object
     workplace_encoded_urban  37.909709    object
     workplace_encoded_rural  37.909709    object
                air_pressure  30.426716   float64
                  wind_speed  30.179344   float64
                    temp_min  29.870130   float64
                    temp_max  29.560915   float64
      marital_status_encoded  23.933210   float64

Group 4: Columns with ≤20% missing data
                      Column  Missing % Data Type
   profession_group_encoded  18.923933   float64
             reason_encoded  16.821274   float64
                        age  12.739641   float64
               time_encoded  12.059369     Int64
          age_group_encoded   7.421150     Int64
                   latitude   6.493506   float64
                  longitude   6.493506   float64
             method_encoded   2.906617   float64
```

```
                suicide_month   2.411874     Int64
                  suicide_day   2.411874     Int64
    suicide_weekday_encoded   2.411874     Int64
               suicide_week   2.411874     Int64
               suicide_year   2.411874     Int64
  religion_encoded_buddhist   2.411874    object
    religion_encoded_muslim   2.411874    object
religion_encoded_indigenous   2.411874    object
     religion_encoded_hindu   2.411874    object
 religion_encoded_christian   2.411874    object
          hometown_encoded   1.669759   float64
gender_encoded_third gender   0.432900    object
      gender_encoded_female   0.432900    object
        gender_encoded_male   0.432900    object
```

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format

# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename = '5_encoded.xlsx'

# # Save the DataFrame to an Excel file
# df_encoded.to_excel(filename, index=False)

# # Download the file
# files.download(filename)
```

Bottom